

**UNIVERSIDAD PRIVADA DE TACNA
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA DE SISTEMAS**



EXAMEN PRÁCTICO DE LA UNIDAD I

CURSO:
SI-988 SOLUCIONES MÓVILES II

AUTOR:
2022073503 – SERGIO COLQUE PONCE

URL GITHUB:
<https://github.com/diegocastillo12/fixtrack-mobile>

TIPO DE EXAMEN: 1

DOCENTE:
DR. OSCAR JUAN JIMENEZ FLORES

TACNA — PERÚ

Indicaciones

- El docente le asigna un único tipo de examen (1, 2, 3 o 4). Desarrolle exclusivamente el tipo asignado y márkelo claramente en la carátula.
- Trabjará sobre la app móvil construida en clase correspondiente a su Sprint 1, la cual debe estar desplegada y emulada en su estación asignada. El login ya debe encontrarse funcional; si no cuenta con backend propio, utilice el servicio en la nube ya integrado (Firebase, Supabase, MockAPI o API REST). La calificación se centra rigurosamente en el código cliente (Mobile App).
- La evidencia en el Aula Virtua, será subir un archivo en formato PDF nombrado obligatoriamente SI988-EXPRAC-U1-ApellidoNombre.pdf
- El documento debe incluir el desarrollo paso a paso de cada ítem de su tipo de examen, adjuntando capturas legibles de su entorno de desarrollo/terminal y una breve explicación técnica de lo implementado.
- Suba todo el código fuente con los cambios implementados a su repositorio individual de GitHub. En el archivo README.md detalle la lista de cambios y las secciones/archivos específicos del proyecto móvil afectados.
- El uso de herramientas de IA está permitido bajo su estricta responsabilidad, no se admitirá atribución de errores a la IA, debiendo el estudiante dominar y explicar técnicamente cada línea de código.
- No copie código de talleres ni de compañeros, las clases, vistas, entidades y paquetes deben guardar estricta correspondencia con su proyecto de Sprint 1. Cualquier detección de plagio o código duplicado anulará la evaluación sin lugar a reclamo.
- *Al concluir su desarrollo, deberá convocar al docente para la sustentación y validación en tiempo real de todos los puntos programados y pruebas ejecutadas en su estación.*

TIPO DE EXAMEN 1. LOGIN, BLOQUEO POR INTENTOS Y CIERRE ANTE HTTP 401

Evaluación práctica sobre robustez de autenticación, control de peticiones y pruebas unitarias (5 ítems - 4 puntos c/u - Total: 20 pts)

1. Validación de formulario y feedback en UI (4 puntos)

Modifique la vista de login de su app móvil para validar las credenciales antes de procesar el envío. Si el campo de usuario o contraseña se encuentra vacío o no cumple un formato mínimo válido (por ejemplo, email con estructura incorrecta o contraseña con longitud menor a 6 caracteres), el botón de envío debe inhabilitarse o mostrar un mensaje descriptivo de error inline sin ejecutar la llamada al servicio. Al ingresar credenciales que cumplan la validación, la interfaz debe proporcionar retroalimentación visual inmediata activando un estado de carga (loading spinner o texto deshabilitado) mientras se procesa la solicitud.

Evidencia en PDF: Captura del código fuente con la validación y el control del estado de carga, captura de pantalla del emulador mostrando el error visible en la UI ante datos inválidos, y captura del estado de carga (loading) activo durante la autenticación.

Archivos: app/lib/features/auth/presentation/login_page.dart

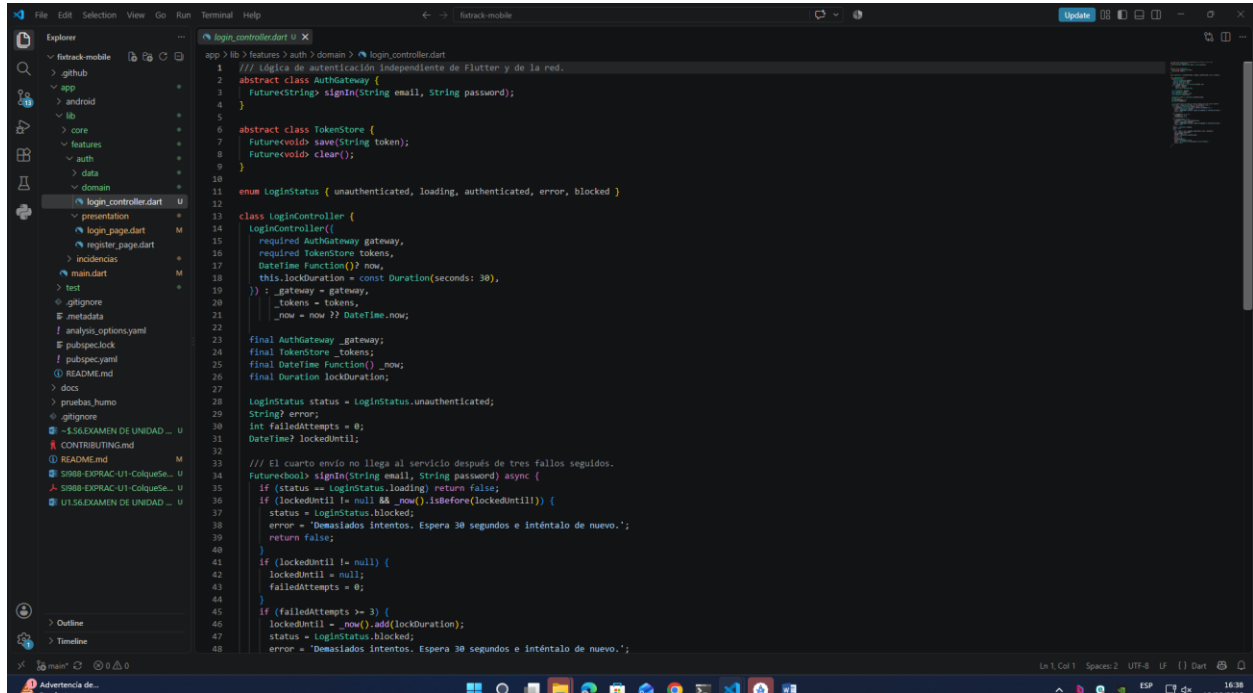
```
login_page.dart M X
app > lib > features > auth > presentation > login_page.dart
1 import 'package:flutter/material.dart';
2 import 'package:supabase_flutter/supabase_flutter.dart';
3
4 import '../data/supabase_auth_gateway.dart';
5 import '../domain/login_controller.dart';
6 import '../incidencias/presentation/incidencias_page.dart';
7 import 'register_page.dart';
8
9 class LoginPage extends StatefulWidget {
10   const LoginPage({super.key});
11
12   @override
13   State<LoginPage> createState() => _LoginPageState();
14 }
15
16 class _LoginPageState extends State<LoginPage> {
17   final _formKey = GlobalKey<FormState>();
18   final _emailController = TextEditingController();
19   final _passwordController = TextEditingController();
20
21   bool _passwordVisible = false;
22   bool _isLoading = false;
23   String? _errorMessage;
24   late final LoginController _login = LoginController(
25     gateway: SupabaseAuthGateway(),
26     tokens: LocalTokenStore(),
27   );
28
29   @override
30   void dispose() {
31     _emailController.dispose();
32     _passwordController.dispose();
33     super.dispose();
34   }
35
36   Future<void> _iniciarSesion() async {
37     if (_isLoading) return;
38     FocusManager.instance.primaryFocus?.unfocus();
39     if (!_formKey.currentState!.validate()) return;
40
41     setState(() {
42       _isLoading = true;
43       _errorMessage = null;
44     });
45     final success = await _login.signIn(
46       _emailController.text,
47       _passwordController.text,
48     );
49   }
```

2. Programe el bloqueo de intentos de autenticación (4 puntos)

Implemente en el código de su aplicación (en la capa de lógica de negocio o ViewModel, no como texto cosmético en la vista) el control de tres logins fallidos consecutivos. Al registrarse el cuarto intento consecutivo, la app debe bloquearse, es decir no debe emitir ninguna llamada al servicio de autenticación y presentará un mensaje de bloqueo temporal explícito al usuario.

Evidencia en PDF: Identificación clara del archivo y bloque de la función donde se administra el contador de intentos fallidos y la condición de interrupción previa a la llamada del servicio.

Archivos: app/lib/features/auth/domain/login_controller.dart



```
1 // Lógica de autenticación independiente de Flutter y de la red.
2 abstract class AuthGateway {
3   Future<String> signIn(String email, String password);
4 }
5
6 abstract class TokenStore {
7   Future<void> save(String token);
8   Future<void> clear();
9 }
10
11 enum LoginStatus { unauthenticated, loading, authenticated, error, blocked }
12
13 class LoginController {
14   LoginController({
15     required AuthGateway gateway,
16     required TokenStore tokens,
17     DateTime Function()? now,
18     this.lockDuration = const Duration(seconds: 30),
19   }) : _gateway = gateway,
20       _tokens = tokens,
21       _now = now ?? DateTime.now;
22
23   final AuthGateway _gateway;
24   final TokenStore _tokens;
25   final DateTime Function() _now;
26   final Duration lockDuration;
27
28   LoginStatus status = LoginStatus.unauthenticated;
29   String? error;
30   int failedAttempts = 0;
31   DateTime? lockedUntil;
32
33   /// El cuarto envío no llega al servicio después de tres fallos seguidos.
34   Future<bool> signIn(String email, String password) async {
35     if (status == LoginStatus.loading) return false;
36     if (lockedUntil != null && _now().isBefore(lockedUntil)) {
37       status = LoginStatus.blocked;
38       error = "Demasiados intentos. Espera 30 segundos e inténtalo de nuevo.";
39       return false;
40     }
41     if (lockedUntil != null) {
42       lockedUntil = null;
43       failedAttempts = 0;
44     }
45     if (failedAttempts >= 3) {
46       lockedUntil = _now().add(lockDuration);
47       status = LoginStatus.blocked;
48       error = "Demasiados intentos. Espera 30 segundos e inténtalo de nuevo.";
```

3. Programe el cierre de sesión ante error HTTP 401 Unauthorized (4 puntos)

Configure en el cliente HTTP (o mediante un interceptor desacoplado/simulable) el manejo reactivo ante respuestas con código de estado HTTP 401. Cuando una petición que viaja con sesión activa reciba dicho código, debe eliminar de inmediato el token de autenticación del almacenamiento local, forzar el estado de sesión cerrada en la aplicación y bloquear cualquier reintento automático de esa petición.

Evidencia en PDF: Captura del archivo y función del interceptor (es un fragmento de código que actúa como un puente o "filtro" para las peticiones HTTP) / middleware donde se captura el código 401, se purga el almacenamiento local y se redirecciona o reinicia la sesión.

Archivos: app/lib/core/network/dio_client.dart;
app/lib/features/incidencias/data/incidencia_repository_impl.dart

```
dio_client.dart M X
app > lib > core > network > dio_client.dart
1 import 'dart:async';
2 import 'dart:developer' as developer;
3
4 import 'package:dio/dio.dart';
5
6 class DioClient {
7   DioClient({
8     required String baseUrl,
9     required Future<String?> Function() tokenProvider,
10    Future<void> Function()? onUnauthorized,
11  }) {
12    dio = Dio(
13      BaseOptions(
14        baseUrl: baseUrl,
15        connectTimeout: const Duration(seconds: 10),
16        receiveTimeout: const Duration(seconds: 10),
17        sendTimeout: const Duration(seconds: 10),
18        headers: {
19          'Accept': 'application/json',
20          'Content-Type': 'application/json',
21        },
22      ),
23    );
24
25    // Interceptor de autenticación.
26    dio.interceptors.add(
27      InterceptorsWrapper(
28        onRequest: (options, handler) async {
29          final token = await tokenProvider();
30
31          if (token != null && token.isNotEmpty) {
32            options.headers['Authorization'] = 'Bearer $token';
33          }
34
35          handler.next(options);
36        },
37      ),
38    );
39
40    // Interceptor de registro seguro.
41    dio.interceptors.add(
42      LogInterceptor(
43        requestBody: true,
44        responseBody: false,
45        requestHeader: false,
46        responseHeader: false,
47        error: true,
48        logPrint: (object) {
```

```
incidencia_repository_impl.dart M X
app > lib > features > incidencias > data > incidencia_repository_impl.dart
1 import 'package:dio/dio.dart';
2 import 'package:flutter/material.dart';
3 import 'package:uuid/uuid.dart';
4 import 'package:supabase_flutter/supabase_flutter.dart';
5
6 import '../core/navigation/app_navigator.dart';
7 import '../core/network/dio_client.dart';
8 import '../auth/data/supabase_auth_gateway.dart';
9 import '../auth/presentation/login_page.dart';
10 import '../domain/incidencia.dart';
11 import '../domain/incidencia_repository.dart';
12 import 'incidencia_dto.dart';
13 import 'incidencia_mapper.dart';
14
15 class IncidenciaRepositoryImpl implements IncidenciaRepository {
16   IncidenciaRepositoryImpl({Dio? dio}) : _dio = dio ?? _creareDio();
17
18   final Dio _dio;
19
20   static Dio _creareDio() {
21     const baseUrl = String.fromEnvironment(
22       'API_BASE',
23       defaultValue: 'http://10.0.2.2:4010',
24     );
25
26     final cliente = DioClient(
27       baseUrl: baseUrl,
28       tokenProvider: () async {
29         return Supabase.instance.client.auth.currentSession?.accessToken;
30       },
31       onUnauthorized: () async {
32         await LocalTokenStore().clear();
33         await Supabase.instance.client.auth.signOut(scope: SignOutScope.local);
34         appNavigatorKey.currentState?.pushAndRemoveUntil(
35           MaterialPageRoute<void>(builder: (_) => const LoginPage()),
36           (_) => false,
37         );
38       },
39     );
40
41     return cliente.dio;
42   }
43
44   @override
45   Future<List<Incidencia>> obtenerIncidencias() async {
46     final response = await _dio.get<dynamic>{
47       '/incidencias',
48       queryParameters: {'limit': 20},
49     };
50   }
51 }
```

4. Ejecute tres pruebas unitarias del login en la terminal (4 puntos)

Diseñe y ejecute tres pruebas unitarias por consola sin necesidad de emulador ni red externa bajo las siguientes condiciones.

- (a) Estado inicial: verificar que al instanciar el servicio/caso de uso no existe sesión previa activa.
- (b) Autenticación exitosa: ante credenciales válidas, el estado pasa a autenticado y el token recibido se almacena correctamente.
- (c) Autenticación rechazada: ante respuesta errónea del backend, el estado refleja error y se asegura que no se reintente la llamada.

Evidencia en PDF: Captura del terminal con el comando exacto ejecutado y la transcripción/captura completa de la salida generada por el test runner.

Archivos: app/test/examen_tipo1_test.dart



5. Ejecute dos pruebas unitarias del bloqueo y de la respuesta 401 (4 puntos)

Diseñe y ejecute por consola dos pruebas unitarias focalizadas.

(a) Verificación de bloqueo: tras acumular tres respuestas fallidas consecutivas, demostrar mediante un mock/spy que el servicio de autenticación no recibe invocaciones en el cuarto intento (zero calls).

(b) Verificación de HTTP 401: inyectar un código 401 simulado y comprobar que el token queda totalmente limpio/nulo, registrándose un único envío sin reintentos.

Evidencia en PDF: Código fuente completo de las dos pruebas unitarias y captura de pantalla de la terminal con las pruebas en estado aprobado (PASS / verde).

Archivos: app/test/examen_tipo1_test.dart

Ejecución de pruebas

Desde la raíz del repositorio:

```
cd app
```

```
flutter test test/examen_tipo1_test.dart
```

Código completo de las pruebas

```
import 'dart:convert';
import 'dart:typed_data';

import 'package:app/core/network/dio_client.dart';
import 'package:app/features/auth/domain/login_controller.dart';
import 'package:dio/dio.dart';
import 'package:flutter_test/flutter_test.dart';

void main() {
  group('Examen Tipo 1', () {
    test('a: estado inicial sin sesión', () {
      final login = LoginController(gateway: _AuthFake(), tokens: _TokensFake());
      expect(login.status, LoginStatus.unauthenticated);
      expect(login.failedAttempts, 0);
    });

    test('b: login exitoso guarda token y autentica', () async {
      final auth = _AuthFake();
      final tokens = _TokensFake();
      final login = LoginController(gateway: auth, tokens: tokens);
      expect(await login.signIn('usuario@upt.pe', 'secreto'), isTrue);
      expect(auth.calls, 1);
      expect(auth.lastEmail, 'usuario@upt.pe');
      expect(login.status, LoginStatus.authenticated);
      expect(tokens.token, 'token-de-prueba');
    });

    test('c: login rechazado registra error sin reintento', () async {
      final auth = _AuthFake(reject: true);
      final login = LoginController(gateway: auth, tokens: _TokensFake());
      expect(await login.signIn('usuario@upt.pe', 'invalida'), isFalse);
      expect(login.status, LoginStatus.error);
      expect(auth.calls, 1);
    });

    test('d: el cuarto intento bloqueado genera cero llamadas', () async {
      final auth = _AuthFake(reject: true);
      final login = LoginController(gateway: auth, tokens: _TokensFake());
      for (var i = 0; i < 3; i++) {
        expect(await login.signIn('usuario@upt.pe', 'invalida'), isFalse);
      }
      final callsBeforeFourth = auth.calls;
      expect(await login.signIn('usuario@upt.pe', 'invalida'), isFalse);
      expect(auth.calls - callsBeforeFourth, 0);
      expect(login.status, LoginStatus.blocked);
      expect(login.error, contains('Espera 30 segundos'));
    });

    test('e: HTTP 401 borra token y no reintenta', () async {
      final tokens = _TokensFake()..token = 'token-de-prueba';
      var sessionActive = true;
      final client = DioClient(
        baseUrl: 'http://localhost:4010',
        tokenProvider: () async => sessionActive ? tokens.token : null,
        onUnauthorized: () async {
          await tokens.clear();
          sessionActive = false;
        },
      );
    });
  });
}
```



```
    );
    final adapter = _UnauthorizedAdapter();
    client.dio.httpClientAdapter = adapter;
    await expectLater(client.dio.get<dynamic>('/recurso'),
        throwsA(isA<DioException>()));
    expect(tokens.token, isNull);
    expect(sessionActive, isFalse);
    expect(adapter.calls, 1);
    expect(adapter.hadAuthorization, isTrue);
  });
});
}

class _AuthFake implements AuthGateway {
  _AuthFake({this.reject = false});
  final bool reject;
  int calls = 0;
  String? lastEmail;

  @override
  Future<String> signIn(String email, String password) async {
    calls++;
    lastEmail = email;
    if (reject) throw StateError('Credenciales rechazadas');
    return 'token-de-prueba';
  }
}

class _TokensFake implements TokenStore {
  String? token;

  @override
  Future<void> save(String value) async {
    token = value;
  }

  @override
  Future<void> clear() async {
    token = null;
  }
}

class _UnauthorizedAdapter implements HttpClientAdapter {
  int calls = 0;
  bool hadAuthorization = false;

  @override
  Future<ResponseBody> fetch(RequestOptions options,
      Stream<Uint8List>? requestStream, Future<void>? cancelFuture) async {
    calls++;
    hadAuthorization = options.headers.containsKey('Authorization');
    return ResponseBody.fromString(jsonEncode({'error': 'unauthorized'}), 401,
        headers: {'content-type': ['application/json']});
  }

  @override
  void close({bool force = false}) {}
}
```